

ITMO University
Software Engineering

Архитектура ЭВМ
(аппаратное обеспечение вычислительных систем)

Лектор: Скаков П. С.

1st Year, Spring Semester

Написан: Коткин Михаил, M3100

Содержание

1. Представление чисел	2
1.1. Беззнаковые целые числа	2
1.2. Знаковые целые числа	2
1.2.1. Бит под знак	2
1.2.2. Сдвиг-128 и сдвиг-127	3
1.2.3. Дополнение до двух	3
1.2.4. Дополнение до одного	4
1.2.5. Форма с чередованием	4
1.2.6. Система счисления с основанием -2	4
1.2.7. Симметричная троичная система	4
1.3. Числа с фиксированной точкой	5
1.3.1. Округление	5
1.4. Числа с плавающей точкой	6
1.4.1. Специальные числа	6

1. Представление чисел

Информацию в компьютере удобно представлять в виде 0 и 1 так как на физическом уровне гораздо легче создать устройство, которое различает всего два состояния (есть ток — 1, нет тока — 0)

1.1. Беззнаковые целые числа

Просто переводим число в двоичную систему счисления и всё... (Напомним, что для перевода нам нужно вычитать из числа максимально возможные степени двойки)

Рассмотрим для примера 8-битное целое беззнаковое число(диапазон 0...255):

128	64	32	16	8	4	2	1
0	0	0	0	0	1	0	1
7	6	5	4	3	2	1	0
Старший бит							Младший бит

Где в клеточках - двоичные знаки, над клеточками - их *веса*, а под клеточками - номера (они начинаются с нуля так как тогда удобнее сопоставлять их с весами - $2^{\text{номер клеточки}}$). Для перевода в 10-ичную систему счисления надо умножить двоичные знаки на веса и сложить их.

Вспомним определение байта. **Байт** - это наименьшая нумеруемая ячейка памяти.

Так же стоит отметить, что не очень правильно говорить первый и последний бит так как наименьшая индексируемая ячейка памяти - это байт, а какой бит в нём первый, какой последний - зависит от конкретного устройства. Стоит использовать терминологию *младший и старший биты*

1.2. Знаковые целые числа

Вот тут есть достаточно много способов хранения, мы рассмотрим 7.

1.2.1. Бит под знак

При таком кодировании один бит выделяется под знак, оставшиеся биты хранят модуль числа. Рассмотрим 8-битное знаковое число закодированное в такой форме:

	64	32	16	8	4	2	1
+/-	0	0	0	0	1	0	1
7	6	5	4	3	2	1	0

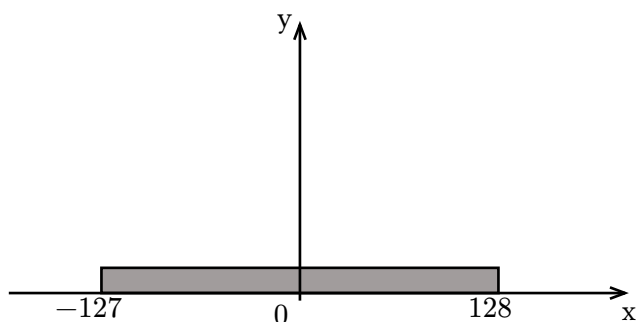
Надо решить какой из знаков + и - будет кодироваться 0, а какой 1. Обычно + кодируется 0, чтобы запись положительных чисел совпадала с беззнаковыми. У такого кодирования есть проблема, у нас есть два нуля:

+0(0b00000000) и -0(0b10000000). Получается для сравнения чисел в такой кодировке надо сравнить числа побитово и если они не совпали проверить не +0 и -0 ли это. Кроме того из-за этого сужается диапазон, он становится -127...127

1.2.2. Сдвиг-128 и сдвиг-127



Диапазон 8-битного беззнакового числа



Диапазон при сдвиге-127

Вспомним диапазон целого беззнакового числа(рис:1). А теперь сдвинем точку отсчёта и получим вторую картинку. В зависимости от того хотим мы больше на одно число положительных или отрицательных чисел в нашем диапазоне мы делаем сдвиг-128 или сдвиг-127. Вообще-то мы храним числа как беззнаковые просто раскодировав, вычитаем из числа 127 или 128. Например, $5 = 10000101_2 = 128 + 5$

1.2.3. Дополнение до двух

В дополнении до 2, вес старшего бита в числе становится отрицательным, вот пример:

-128	64	32	16	8	4	2	1
1	1	1	1	1	0	1	1
7	6	5	4	3	2	1	0

Тут, например, закодировано число $-5 = -128 + 64 + 32 + 16 + 8 + 2 + 1$

Как такое быстро посчитать? $-x = \sim x + 1$, где $\sim x$ это просто инвертирование всех битов числа

- В случае 8-битных чисел мы имеем цельный диапазон $[-128; 127]$ без дубликатов и дырок
- При таком кодировании у отрицательных чисел первый бит - 1, у положительных - 0. Но это просто так удачно получилось, а не мы на основе этого кодировали числа как в случае бита под знак.
- В такой форме вычитание чисел это просто сложение с числом обратного знака(то есть на уровне железа достаточно поддерживать одну операцию).
- При выходе за границы диапазона срабатывает *модулярная арифметика* то есть при выходе за один конец диапазона мы оказываемся с другого конца. Это не единственный вариант обработки таких ситуаций: арифметика с насыщением(при переполнении ставится максимальное или минимальное значение), выбрасывание ошибки

В языке C по стандарту гарантируется при переполнении беззнаковых типов – использование модулярной арифметики, а переполнение знаковых типов – Undefined Behavior. Почему? Потому что раньше железо было разное и знаковые числа хранились по-разному поэтому стандарт ничего и не говорил, что делать при переполнении знаковых чисел, а беззнаковые везде хранились одинаково. Сейчас большинство компьютеров хранят знаковые числа в дополнении до двух. Поэтому требование, чтобы знаковые числа хранились так даже было бодавлено в стандартах c23 и c++20. Однако UB всё ещё не убрали...

Кстати из-за возможности разного хранения знаковых чисел определения `int` по стандарту с – это целосленный знаковый тип, диапазон которого не хуже чем $[-(2^{16} - 1); 2^{16} - 1]$ Пусть у нас есть число, закодированное в дополнении до двух, попытаемся найти его модуль:

```
1 int x;
```

```
2 int x_abs;
```

```

3 if (x >= 0) x_abs = x;
4 else x_abs = -x; // а если x = -128
   переполнение и UB

```

```

1 int x;
2 uint x_abs;
3 if (x >= 0) x_abs = x;
4 else x_abs = -(uint)x; // срабатывает
   модулярная арифметика

```

Ещё один момент, где можно столкнуться с интересными последствиями такого UB:

```

1 for (int x = 5; x > 0; x++)
2     printf("%i\n", x);

```

Так как числа хранятся в дополнении до двух, то мы знаем, что когда мы достигнем максимального значения, следующим будет минимальное и цикл завершится. Но для компилятора переполнение знаковых целых чисел - UB и он понимает это как делаю, что хочу. Он проявляет “нездоровый интеллект” и убирает наше условие $x > 0$ так как на его взгляд оно всегда выполняется. И такой цикл превращается в бесконечный

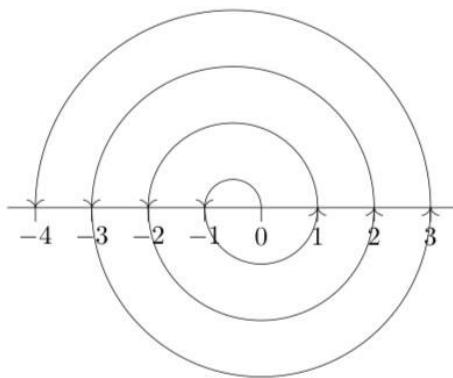
1.2.4. Дополнение до одного

-127	64	32	16	8	4	2	1
1	1	1	1	1	0	1	1
7	6	5	4	3	2	1	0

В такой форме $-x$ это просто инверсия всех битов. Однако при таком кодировании два нуля :(

1.2.5. Форма с чередованием

Зачем? Можно масштабировать число нулями слева: 000 число, например, в дополнении до двух с отрицательными числами так не получится



	Код	Значение
0	000	0
1	001	-1
2	010	1
3	011	-2
4	100	2
5	101	-3

Замечание: младший бит - отвечает за знак, оставшиеся - модуль числа

1.2.6. Система счисления с основанием -2

-128	64	-32	16	-8	4	-2	1
1	1	1	1	1	0	1	1
7	6	5	4	3	2	1	0

Диапазон: $[-170; 85]$ - отрицательных чисел в два раза больше чем положительных

Если количество бит в числе чётное, то отрицательных чисел в два раза больше чем положительных, иначе - наоборот.

1.2.7. Симметричная троичная система

Будем кодировать числа в троичной системе. Тут у нас не биты, а *триты* (не байт, а *трайт*): 0, 1, 2 или -1, 0, 1 их часто записывают так: $z, 0, 1$

81	27	9	3	1
0	0	1	z	z
4	3	2	1	0

Тут закодировано число 5, а $-5 = 00z11_3$ кодируется так.

1.3. Числа с фиксированной точкой

Числа с фиксированной точкой - это способ зранения дробных чисел в котором под целую и дробную часть выделено фиксированное число бит. В железе нет специальных инструкций для работы с таким типом хранения, так как его легко можно эмулировать, используя целочисленные типы.

В таком случае мы работаем в форме со сдвигом, чтобы получить настоящее число нам надо поделить закодированное на $2^{\text{количество битов}}$. Мы уже встречались с таким(проценты, 5% - 0.05)

Рассмотрим число в формате 8.8:

-128	64	-32	16	-8	4	-2	1	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}
0	0	0	0	0	0	1	1	1	0	1	0	0	0	0	0

Тут закодировано число 3.625. А как мы это получили?

Целую часть переводим как просто целое число(например дополнение до 2). Дробную часть переводим по алгоритму: домножаем число на 2 и смотри на старший разряд, нолики можно откидывать

0	625
1	250
1	0

А теперь попробуем перевести 0.1 в формат 8.8: Получим $00000000.0(0011)_2$ получили бесконечную периодическую дробь...

Есть такая теорема: Конечная десятичная дробь переходит в конечную или периодическую двоичную.

Но как нам всё-таки засунуть бесконечную двоичную дробь в формат 8.8? Нужно округление

1.3.1. Округление

- вверх($k + \infty$)
- вниз($k - \infty$)
- к нулю(просто обрезаем лишние биты)

В первых трёх способах мы можем получить ошибку почти в 1 последнего разряда

- школьное округление: если цифра: 0, 1, 2, 3, 4 – вниз от нуля (в 2 числах: 0)
если цифра: 5, 6, 7, 8, 9 – вверх от нуля (в 2 числах: 1)

Проблема такого округления в *направленной погрешности*: все положительные числа вида а.5 будут округляться в большую сторону, с отрицательными наоборот (мы хотим, чтобы погрешности в таком случае не накапливались, а компенсировали друг друга)

Пример: среднее арифметическое чисел 1.5, 2.5, ..., 10.5 если их вначале округлить будет больше чем, если считать не округляя.

- округление к ближайшему чётному: Округляем к ближайшему числу, в случае если число посередине, то к ближайшему чётному (это позволяет компенсировать погрешность, а не накапливать ее)
- (существует округление к ближайшему нечётному)

А как умножать числа с фиксированной точкой?

```
1 uint x, y, z; // x = X * 2^16, y = Y * 2^16
2 z = (uint64_t)(x * y) / (1 << 16); // z = (X * 2^16 * Y * 2^16) / 2^16
```

дальше надо округлить z для этого можно поделить целочисленно и посмотреть остаток от деления, по нему применить какое-то из округлений выше.

А как делить числа с фиксированной точкой?

```
1 uint x, y, z; // x = X * 2^16, y = Y * 2^16
2 z = (uint64_t)x *(1 << 16)/y; // z = (X * 2^16) / (Y * 2 ^ 16) * 2^16
```

1.4. Числа с плавающей точкой

Вспомним физику: там мы записывали константы как-то так $1,23456 * 10^7$

Почему это удобно? Во-первых, меньше писать(меньше хранить), во-вторых, так нам легко поддерживать относительную точность.

Относительная точность - количество цифр после первой ненулевой цифры

Например, $1,23456 * 10^7 = 123456000$ но на самом то деле там не нолики, а какие-то цифры, у такого числа относительная точность - 6

Пример: $5.375 = 101.011_2 = 1.01 * 2^2$ здесь мы будем писать умножить на 2^k десятичное число, чтобы не запутаться. Степень 2, k - называется экспонентой.

А как такое хранить?

Существует международный стандарт **IEEE-754**:

- Мантиса хранится в формате бит под знак, а экспонента в форме со сдвигом с округлением вниз...
- Заметим, что все числа начинаются с 1. (кроме 0, но о нём позже) поэтому такой префикс можно не хранить

sign	exp	mantisa
------	-----	---------

Название	всего бит	exp	mantisa	комментарии
single_precision	32	8	23	<code>float</code> в c++ и c
double_precision	64	11	52	<code>double</code> в c++ и c
quadruple_precision	128	15	112	обычное железо такое не поддерживает
half_precision	16	5	10	<code>float16_t</code> в c++ и <code>_Float16</code> в c начиная с c23 и c++23
				соответственно
externad_precision	80	15	64	не общепринятый; здесь 1. включено в мантису

Замечание: Про то что такое `long double` все так и не договорились, поэтому его стоит использовать с осторожностью, понимая, что это на конкретном железе и компиляторе(например в MSVC это `double_precision`)

Закодируем $\frac{1}{10}$ в single precision:

$0.1_{10} \rightarrow 0.00011001100110011001100110 = 1.100110011001100110011001 * 2^{-4}$ сдвигаем -4 на 127, получаем 01111011 получили:

0	01111011	10011001100110011001101
---	----------	-------------------------

1.4.1. Специальные числа

Рассмотрим самое маленькое число в half precision:

$$\begin{array}{rcl} a & = & 1.0000000000 * 2^{-14} \\ b & = & \frac{1.0000000001 * 2^{-14}}{0.0000000001 * 2^{-24}} \end{array}$$

тогда:

- `a != b` `true`
- `a > b` `true`

- `a - b == 0` `true`

То есть у нас есть два не равных числа разность между которыми 0!

1. Денормализованные числа:

Это числа вида ± 0 . мантиса $\times 2^{-14}$. Такие числа заполняют диапазон между нулём и наименьшим нормализованным числом (1.0×2^{-14}) и позволяют нам закодировать разность `a` и `b`.

2. минимальная экспонента(11111)

- мантиса = 0 числа $\pm \infty$ (в зависимости от знака)
- мантиса $\neq 0$ - NaN (not a number)

NaN - это такой “зверёк” у которого нет знака, любая операция, где есть он даёт его же. В случае операции с двумя NaN какой из них выживает зависит от железа, софта. Нужны они для того, чтобы сообщать об ошибках, например вычитание бесконечности, деление 0 на 0 и тд. Кроме того ими можно заполнять память. Зверьки бывают двух видов: тихие(старший бит мантисы 1) - не бросают исключения и сигнальные(старший бит мантисы 0) - бросают исключения.

Зверьки не равны никакому числу даже себе самому тогда проверка на NaN:

```
1 if (a != a) // Это даже быстрее, чем функция isnan(x) в c
2 printf("It is a NaN");
```

Эти исключения на уровне железа, а не как ваши исключения на c++, при них программа не обязательно падает.

$\pm \infty$ Работает как в математике и позволяет делить на 0.0

Есть ещё две операции с дробными числами: MAD multiply and add $a = b * c + d$ (два округления) и FMA - тоже самое, но с одним округлением в конце.

Всякие новые интересные форматы числе с плавающей точкой:

Полезные, например, для вычисления в нейронных сетях - жертвуем точностью и увеличиваем скорость вычислений.

Название	всего бит	exp	mantisa	комментарии
bfloat16	16	8	7	
b8e5m2	8	5	2	
b8e4m3fn	8	4	3	fn - значит выкидываем $\pm \infty$
b8e4m3fnuz				uz - unsigned zero только один ноль
b8e4m3fnuz0				uz0 - ещё и единственный NaN

Для всех этих форматов доступно только матричное умножение

Название	всего бит	exp	mantisa	комментарии
mxe2m1	4	2	1	mx - числа хранятся в блоках по 32 значения с общим множителем в формате e8m0 или блок 16 значений и множитель e4m3fn
nvfp4	4	2	1	